

数据库系统

工程作业

PurpleBank: 核苷酸序列数据库

鲁昕宁 2414015

2026 年 5 月 20 日

<https://github.com/SidneyLu/PurpleBank>

1 项目信息（10 分）

1.1 基本信息

项目名称	PurpleBank: 核苷酸序列数据库
必备环境	Windows 11, MySQL 8.0+, Python 3.10+, BioPython, Node.js 18+

1.2 系统主要功能简介（4 分）

PurpleBank 是一个面向生物信息学研究的核苷酸序列数据库系统，专为序列检索与信息管理设计。主要功能如下：

1. 访客无需登录即可直接进行序列检索与详情查看；
2. 普通用户可提交序列变更请求（CREATE/UPDATE/DELETE），但需经管理员审核后方可生效；
3. 系统通过触发器、存储过程、视图和事务机制保证数据一致性、可审计性与可追踪性；
4. 管理员用户管理同样采用”请求-审核-执行”流程，避免直接越权改动账户数据。

1.3 系统主要页面截图（6 分）

PurpleBank Search New Request My Requests Sequence Reviews User Reviews Users admin (admin) Logout						
PurpleBank Sequence Explorer Guest users can browse sequence data. Mutations require reviewed requests.						
Accession / locus / definition Genus All statuses Search						
13618 records found. Page 1/1362						
Accession	Name	Genus	Status	Length	Updated	Ops
DEMO_CREATE_OK_001	Nicotiana tabacum	Nicotiana	Approved	20	2026/5/20 13:01:38	2
DEMO_DELETE_OK_001	Nicotiana tabacum	Nicotiana	Approved	24	2026/5/20 13:01:38	1
NC_000932	Arabidopsis thaliana	Arabidopsis	Pending	1440	2026/4/18 19:00:56	1
NC_001320	Oryza sativa Japonica Group	Oryza	Pending	1434	2026/4/18 19:00:56	0

Submit Sequence Change Request

All CREATE/UPDATE/DELETE actions must be reviewed by admin before execution.

Action Type

CREATE

Target Accession

e.g. NC_085725

Reason

Why should this change be approved?

Payload JSON

```
{
  "accession": "NC_NEW_000001",
  "version": "1.0",
  "locus": "new_locus",
  "definition": "new nucleotide sequence",
}
```

Sequence Review Console

Admins approve/reject sequence requests. Approved requests execute transactions.

ID	Action	Accession	Status	Requester	Reason	Review
23	UPDATE	DEMO_NO_SUCH_ACCESSION_001	PENDING	10	update fail	comment <button>Approve</button> <button>Reject</button>
22	UPDATE	DEMO_CREATE_OK_001	APPROVED	10	update ok	real-shot run
21	CREATE	DEMO_CREATE_BAD_001	PENDING	10	create fail	comment <button>Approve</button> <button>Reject</button>
20	CREATE	DEMO_CREATE_OK_001	APPROVED	10	create ok	real-shot run
19	DELETE	DEMO_DELETE_OK_001	APPROVED	10	demo delete	real-shot run

User Management

Admins cannot directly mutate users. Actions are submitted as review requests.

Create User Request

Username

Password

Role

user

Reason

Submit create request

2 系统配置（10 分）

2.1 配置步骤（2 分）

2.1.1 数据库配置

1. 后台数据库：MySQL（数据库名：purple_bank）
2. 启动 MySQL 服务，创建数据库并按顺序执行以下脚本：
 - db/purple_bank_01_database.sql
 - db/purple_bank_02_core_schema.sql
 - db/purple_bank_03_extension_schema.sql
 - db/purple_bank_04_indexes.sql
 - db/purple_bank_05_seed_data.sql
 - db/purple_bank_06_triggers.sql
 - db/purple_bank_07_views.sql
 - db/purple_bank_08_procedures.sql
 - db/purple_bank_09_user_change_request.sql
- 3.（可选）执行数据导入脚本，将 XML/FASTA 原始数据导入 **Sequence** 等业务表。

2.1.2 后端与前端配置

1. 后端：Python（FastAPI + SQLAlchemy）
2. 进入 **apps/backend** 目录，安装依赖并配置 **.env** 文件，启动 Uvicorn（默认地址：127.0.0.1:8001）
3. 前端：TypeScript（Next.js + React）
4. 进入 **apps/frontend** 目录，安装依赖并配置 **.env.local** 文件，启动 Next.js（默认地址：127.0.0.1:3000）

2.2 连接串分析（6 分）

后端连接串由 **apps/backend/app/core/config.py** 中的 **sqlalchemy_url** 属性拼接得到：

```
mysql+pymysql:// {MYSQL_USER}:{MYSQL_PASSWORD}@  
{MYSQL_HOST}:{MYSQL_PORT}/{MYSQL_DATABASE}
```

序号	名称	功能说明	取值
1	mysql+pymysql	指定数据库类型与 Python 驱动	mysql+pymysql
2	MYSQL_USER	数据库登录用户名	root
3	MYSQL_PASSWORD	数据库登录密码	root
4	MYSQL_HOST	数据库主机地址	127.0.0.1
5	MYSQL_PORT	数据库端口	3306
6	MYSQL_DATABASE	目标数据库名	purple_bank

表 1: 数据库连接串参数说明

2.3 连接串代码（2 分）

```

1 # apps/backend/app/core/config.py
2 @property
3 def sqlalchemy_url(self) -> str:
4     return f"mysql+pymysql://{self.mysql_user}:{self.mysql_password}" \
5           f"@{self.mysql_host}:{self.mysql_port}/{self.mysql_database}"
6
7 # apps/backend/.env.example
8 MYSQL_HOST=127.0.0.1
9 MYSQL_PORT=3306
10 MYSQL_USER=root
11 MYSQL_PASSWORD=root
12 MYSQL_DATABASE=purple_bank

```

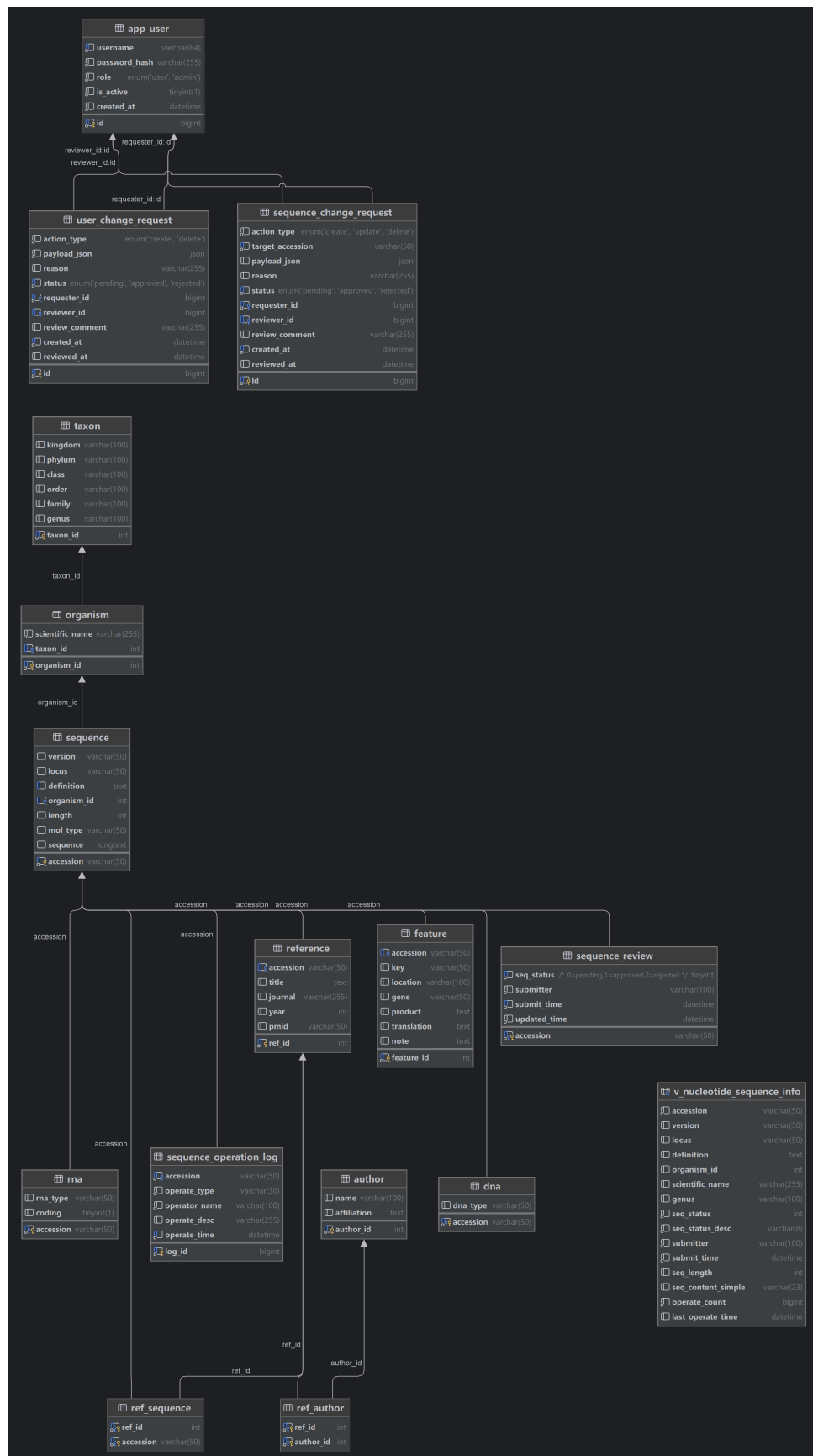
3 数据库设计（14 分）

3.1 数据表（10 分）

创建顺序	数据表名称	主键	参照属性	被参照表及属性
1	Taxon	taxon_id	无	无
2	Organism	organism_id	taxon_id	Taxon(taxon_id)
3	Sequence	accession	organism_id	Organism(organism_id)
4	DNA	accession	accession	Sequence(accession)
5	RNA	accession	accession	Sequence(accession)
6	Feature	feature_id	accession	Sequence(accession)
7	Reference	ref_id	accession	Sequence(accession)
8	Author	author_id	无	无
9	Ref_Sequence	(ref_id, accession)	ref_id, accession	Reference(ref_id), Sequence(accession)
10	Ref_Author	(ref_id, author_id)	ref_id, author_id	Reference(ref_id), Author(author_id)
11	app_user	Id	无	无
12	sequence_review	accession	accession	Sequence(accession)
13	sequence_operation_log	log_id	accession	Sequence(accession)
14	sequence_change_request	Id	requester_id, reviewer_id	app_user(id), app_user(id)
15	user_change_request	Id	requester_id, reviewer_id	app_user(id), app_user(id)

表 2: 数据表结构与关系

3.2 关系图（4 分）



4 含有事务应用的删除操作（13 分）

4.1 功能描述（1 分）

管理员审批通过某条 DELETE 类型序列变更请求后，系统删除目标 accession 在多张业务表中的关联数据，保证该序列数据整体一致地移除。

4.2 涉及的表（2 分）

Ref_Author、Reference、Ref_Sequence、Feature、DNA、RNA、sequence_operation_log、sequence_review、Sequence

4.3 表连接涉及字段（1 分）

Ref_Author.ref_id = Reference.ref_id

4.4 删除条件字段描述（1 分）

字段	规则
Reference.accession	= target.accession
Ref_Sequence.accession	= target.accession
Feature.accession	= target.accession
DNA.accession	= target.accession
RNA.accession	= target.accession
sequence_operation_log.accession	= target.accession
sequence_review.accession	= target.accession
Sequence.accession	= target.accession
sequence_change_request.action_type	必须为 DELETE
sequence_change_request.status	必须为 PENDING 才允许审核执行
sequence_change_request.target_accession	必填，作为跨表删除的统一条件值

表 3: 删除操作条件

4.5 代码（4 分）

```
1 # apps/backend/app/services/sequence_request_service.py
2 try:
3     row = db.execute(text("""
4         SELECT id, action_type, target_accession, payload_json, status
```

```

5         FROM sequence_change_request
6         WHERE id = :request_id
7         FOR UPDATE
8         """), {"request_id": request_id}).first()
9
10        # 状态校验与执行逻辑
11        execution_summary = _execute_sequence_action(
12            db, action_type, target_accession, mutation_payload
13        )
14
15        db.execute(text("""
16            UPDATE sequence_change_request
17            SET status = 'APPROVED', reviewer_id = :reviewer_id,
18                review_comment = :comment, reviewed_at = CURRENT_TIMESTAMP
19            WHERE id = :request_id
20            """), {
21                "reviewer_id": reviewer_id,
22                "comment": comment,
23                "request_id": request_id
24            })
25
26        db.commit()
27    except Exception:
28        db.rollback()
29        raise
30
31    DELETE ra
32    FROM `Ref_Author` AS ra
33    JOIN `Reference` AS r ON r.ref_id = ra.ref_id
34    WHERE r.accession = :accession;
35
36    DELETE FROM `Ref_Sequence` WHERE accession = :accession;
37    DELETE FROM `Reference` WHERE accession = :accession;
38    DELETE FROM `Feature` WHERE accession = :accession;
39    DELETE FROM `DNA` WHERE accession = :accession;
40    DELETE FROM `RNA` WHERE accession = :accession;
41    DELETE FROM `sequence_operation_log` WHERE accession = :accession;
42    DELETE FROM `sequence_review` WHERE accession = :accession;
43    DELETE FROM `Sequence` WHERE accession = :accession;

```

4.6 程序演示（4 分）

4.6.1 API 调用步骤

```

1 # 1) 用户提交 DELETE 请求
2 curl -s -X POST "http://127.0.0.1:8001/api/v1/sequence-requests" \

```

```

3 -H "Authorization: Bearer $USER_TOKEN" \
4 -H "Content-Type: application/json" \
5 -d '{
6     "action_type": "DELETE",
7     "target_accession": "DEMO_DELETE_OK_001",
8     "payload_json": {},
9     "reason": "demo delete"
10 }'
11
12 # 2) 管理员查看请求, 获取 request_id
13 curl -s "http://127.0.0.1:8001/api/v1/admin/sequence-requests" \
14 -H "Authorization: Bearer $ADMIN_TOKEN"
15
16 # 3) 管理员审批通过
17 curl -s -X POST \
18     "http://127.0.0.1:8001/api/v1/admin/sequence-requests/{request_id}/review" \
19     \
20 -H "Authorization: Bearer $ADMIN_TOKEN" \
21 -H "Content-Type: application/json" \
22 -d '{"decision": "APPROVE", "review_comment": "approve delete demo"}'

```

4.6.2 SQL 校验

```

1 USE purple_bank;
2 SELECT COUNT(*) AS c_sequence FROM `Sequence` WHERE accession =
3     'DEMO_DELETE_OK_001';
4 SELECT COUNT(*) AS c_feature FROM `Feature` WHERE accession =
5     'DEMO_DELETE_OK_001';
6 SELECT COUNT(*) AS c_dna FROM `DNA` WHERE accession = 'DEMO_DELETE_OK_001';
7 SELECT COUNT(*) AS c_rna FROM `RNA` WHERE accession = 'DEMO_DELETE_OK_001';
8 SELECT COUNT(*) AS c_ref_seq FROM `Ref_Sequence` WHERE accession =
9     'DEMO_DELETE_OK_001';
10 SELECT COUNT(*) AS c_ref FROM `Reference` WHERE accession =
11     'DEMO_DELETE_OK_001';
12 SELECT COUNT(*) AS c_review FROM `sequence_review` WHERE accession =
13     'DEMO_DELETE_OK_001';
14 SELECT COUNT(*) AS c_log FROM `sequence_operation_log` WHERE accession =
15     'DEMO_DELETE_OK_001';
16
17 -- Ref_Author 通过 Reference.ref_id 关联校验
18 SELECT COUNT(*) AS c_ref_author
19 FROM `Ref_Author` ra
20 JOIN `Reference` r ON r.ref_id = ra.ref_id
21 WHERE r.accession = 'DEMO_DELETE_OK_001';

```

```
{
  "submit": {
    "status": 201,
    "body": "{\"id\":\"19\",\"action_type\":\"DELETE\",\"target_accession\":\"DEMO_DELETE_OK_001\",\"payload_json\":{},\"reason\":\"demo delete\",\"status\":\"PENDING\",\"requester_id\":\"10\",\"reviewer_id\":null,\"review_comment\":null,\"created_at\":\"2026-05-20T13:01:38\",\"reviewed_at\":null}\"",
    "request_id": 19,
    "review": {
      "status": 200,
      "body": "{\"request_id\":\"19\",\"status\":\"APPROVED\",\"execution_summary\":\"DELETE executed for accession DEMO_DELETE_OK_001.\"}"
    },
    "sql": {
      "Sequence": 0,
      "Feature": 0,
      "DNA": 0,
      "RNA": 0,
      "Ref_Sequence": 0,
      "Reference": 0,
      "sequence_review": 0,
      "sequence_operation_log": 0
    }
  }
}
```

14	DELETE	DEMO_DELETE_OK_001	APPROVED	10	demo delete	real-shot run
----	--------	--------------------	----------	----	-------------	---------------

4.6.3 事务回滚演示

```
1 USE purple_bank;
2 -- 先确保样本存在
3 SELECT COUNT(*) AS before_sequence FROM `Sequence`
4 WHERE accession = 'DEMO_DELETE_OK_001';
5
6 START TRANSACTION;
7 -- 模拟删除流程中的前几步
8 DELETE FROM `Feature` WHERE accession = 'DEMO_DELETE_OK_001';
9 DELETE FROM `DNA` WHERE accession = 'DEMO_DELETE_OK_001';
10 -- 人工制造异常（不存在的表）
11 DELETE FROM `TABLE_NOT_EXISTS_FOR_DEMO` WHERE accession =
    ↳ 'DEMO_DELETE_OK_001';
12 -- 这句不会执行；发生错误后执行 ROLLBACK
13 ROLLBACK;
14
15 -- 验证数据已回滚
16 SELECT COUNT(*) AS after_feature FROM `Feature` WHERE accession =
    ↳ 'DEMO_DELETE_OK_001';
17 SELECT COUNT(*) AS after_dna FROM `DNA` WHERE accession =
    ↳ 'DEMO_DELETE_OK_001';
```

```
{
  "before": {
    "feature": 1,
    "dna": 1
  },
  "error": "(1146, \\Table 'purple_bank.table_not_exists_for_demo' doesn't exist\\")",
  "after": {
    "feature": 1,
    "dna": 1
  }
}
```

4.7 备注

在 `Session(autocommit=False)` 模式下，上述所有 SQL 语句位于同一事务中。若任一语句执行异常，`rollback()` 将撤销全部已执行操作，体现了事务的原子性。所有演示请求均可通过“New Request”页面表单发送。

Submit Sequence Change Request

All CREATE/UPDATE/DELETE actions must be reviewed by admin before execution.

Action Type

CREATE

Target Accession

e.g. NC_085725

Reason

Why should this change be approved?

Payload JSON

```
{
  "accession": "NC_NEW_000001",
  "version": "1.0",
  "locus": "new_locus",
  "definition": "new nucleotide sequence",
  "organism_id": 1,
  "mol_type": "DNA",
  "sequence": "ATCGATCGATCGATCGATCG"
}
```

Submit request

5 触发器控制下的添加操作（20 分）

5.1 功能描述（1 分）

管理员审批通过 CREATE 类型请求后，系统向 Sequence 表插入新序列；触发器负责在数据库层进行合法性校验并自动补齐衍生数据。

5.2 触发器描述（2 分）

- TRG_before_insert_sequence_validate:
插入前校验 organism_id 非空且存在、序列长度和字符集合法，并自动计算序列长度；
- TRG_after_insert_sequence:
插入后自动写入 sequence_review 与 sequence_operation_log 表，保证审核状态与审计日志的完整性。

5.3 涉及的表（1 分）

Sequence、Organism、sequence_review、sequence_operation_log

5.4 输入数据（2 分）

字段	规则
accession	必填，主键，不可重复
organism_id	必填，且必须存在于 Organism.organism_id
sequence	必填，去除空白后长度 ≥ 10
sequence	仅允许 IUPAC 核苷酸字符：ATCGUNRYSKWMBDHV

表 4: 序列插入输入规则

5.5 插入操作源码（3 分）

```
1 INSERT INTO `Sequence` (  
2     accession, version, locus, definition, organism_id, mol_type,  
3     sequence  
4 ) VALUES (  
5     :accession, :version, :locus, :definition, :organism_id, :mol_type,  
6     :sequence  
7 );
```

5.6 触发器源码 (3 分)

```
1 CREATE TRIGGER TRG_before_insert_sequence_validate
2 BEFORE INSERT ON `Sequence`
3 FOR EACH ROW
4 BEGIN
5     IF NEW.organism_id IS NULL THEN
6         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT =
7             ↳ 'organism_id is required';
8     END IF;
9
10    IF NOT EXISTS (SELECT 1 FROM `Organism` WHERE organism_id =
11        ↳ NEW.organism_id) THEN
12        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT =
13            ↳ 'organism_id does not exist';
14    END IF;
15
16    IF NEW.sequence IS NULL OR CHAR_LENGTH(TRIM(NEW.sequence)) < 10 THEN
17        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT =
18            ↳ 'sequence content must be non-empty and length >= 10';
19    END IF;
20
21    -- 修正原文档逻辑错误: 添加 NOT, 匹配到非法字符时抛出错误
22    IF NOT REGEXP_LIKE(UPPER(NEW.sequence), '^[ATCGUNRYSKWMBDHV]+$') THEN
23        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT =
24            ↳ 'sequence contains invalid nucleotide characters (IUPAC expected)'
25            ↳ ;
26    END IF;
27
28    SET NEW.length = CHAR_LENGTH(NEW.sequence);
29 END;
30
31 CREATE TRIGGER TRG_after_insert_sequence
32 AFTER INSERT ON `Sequence`
33 FOR EACH ROW
34 BEGIN
35     INSERT INTO `sequence_review`(accession, seq_status, submitter)
36     VALUES (NEW.accession, 1, 'approval_engine')
37     ON DUPLICATE KEY UPDATE
38         seq_status = VALUES(seq_status),
39         submitter = VALUES(submitter),
40         updated_time = CURRENT_TIMESTAMP;
41
42     INSERT INTO `sequence_operation_log`(accession, operate_type,
43         operator_name, operate_desc)
44     VALUES (NEW.accession, 'insert', 'approval_engine',
```

```
15         CONCAT('new sequence inserted (organism_id=', NEW.organism_id,
    ↳      '))');
16 END;
```

5.7 程序演示（8 分）

5.7.1 合法插入演示（4 分）

```
1 # 1) 提交 CREATE 请求
2 curl -s -X POST "http://127.0.0.1:8001/api/v1/sequence-requests" \
3 -H "Authorization: Bearer $USER_TOKEN" \
4 -H "Content-Type: application/json" \
5 -d '{
6     "action_type": "CREATE",
7     "target_accession": "DEMO_CREATE_OK_001",
8     "payload_json": {
9         "accession": "DEMO_CREATE_OK_001",
10        "version": "1.0",
11        "locus": "DEM_LOCUS_CREATE_0",
12        "definition": "Demo valid create",
13        "organism_id": 13596,
14        "mol_type": "DNA",
15        "sequence": "ATCGATCGATCGATCGATCG"
16    },
17    "reason": "demo create success"
18 }'
19
20 # 2) 管理员审批通过
21 curl -s -X POST \
    ↳ "http://127.0.0.1:8001/api/v1/admin/sequence-requests/{request_id}/review" \
    ↳ \
    ↳ \
22 -H "Authorization: Bearer $ADMIN_TOKEN" \
23 -H "Content-Type: application/json" \
24 -d '{"decision": "APPROVE", "review_comment": "approve valid create"}'
```

5.7.2 非法插入演示（4 分）

```
1 # 非法 sequence: 含 X/Z/数字, 不满足 IUPAC 标准
2 curl -s -X POST "http://127.0.0.1:8001/api/v1/sequence-requests" \
3 -H "Authorization: Bearer $USER_TOKEN" \
4 -H "Content-Type: application/json" \
5 -d '{
6     "action_type": "CREATE",
7     "target_accession": "DEMO_CREATE_BAD_001",
```

```
{
  "submit": {
    "status": 201,
    "body": "{\n\"id\":20,\n\"action_type\":\n\"CREATE\",\n\"target_accession\":\n\"DEMO_CREATE_OK_001\",\n\"payload_json\":\n{\n\"locus\":\n\"DEMO_LOCUS_CREATE_OK\",\n\"version\":\n\"1.0\",\n\"mol_type\":\n\"DNA\",\n\"sequence\":\n\"ATCGATCGATCGATCGATCG\",\n\"accession\":\n\"DEMO_CREATE_OK_001\",\n\"definition\":\n\"Demo valid create\",\n\"organism_id\":13596,\n\"reason\":\n\"create ok\",\n\"status\":\n\"PENDING\",\n\"requester_id\":10,\n\"reviewer_id\":null,\n\"review_comment\":null,\n\"created_at\":\n\"2026-05-20T13:01:38\",\n\"reviewed_at\":null}\n}"
  },
  "request_id": 20,
  "review": {
    "status": 200,
    "body": "{\n\"request_id\":20,\n\"status\":\n\"APPROVED\",\n\"execution_summary\":\n\"CREATE executed for accession DEMO_CREATE_OK_001.\n}"
  },
  "sequence": [],
  "review_row": [],
  "log": []
}
```

15	CREATE	DEMO_CREATE_OK_001	APPROVED	10	create ok	real-shot run
----	--------	--------------------	----------	----	-----------	---------------

```
8   "payload_json": {
9     "accession": "DEMO_CREATE_BAD_001",
10    "version": "1.0",
11    "locus": "DEM_LOCUS_CREATE_A",
12    "definition": "Demo invalid create",
13    "organism_id": 13596,
14    "mol_type": "DNA",
15    "sequence": "ACGXYZ123"
16  },
17  "reason": "demo create should fail"
18 }'
```

19 # 管理员审批通过时触发触发器报错

```
21 curl -s -X POST ]
   ↪ "http://127.0.0.1:8001/api/v1/admin/sequence-requests/{request_id}/review" ]
   ↪
   ↪ \
22 -H "Authorization: Bearer $ADMIN_TOKEN" \
23 -H "Content-Type: application/json" \
24 -d ]
   ↪ '{"decision": "APPROVE", "review_comment": "approve invalid create for test"}'
```

```
{
  "submit": {
    "status": 201,
    "body": "{\n\"id\":21,\n\"action_type\":\n\"CREATE\",\n\"target_accession\":\n\"DEMO_CREATE_BAD_001\",\n\"payload_json\":\n{\n\"locus\":\n\"DEMO_LOCUS_CREATE_BAD\",\n\"version\":\n\"1.0\",\n\"mol_type\":\n\"DNA\",\n\"sequence\":\n\"ATCGXYZ123\",\n\"accession\":\n\"DEMO_CREATE_BAD_001\",\n\"definition\":\n\"Demo\ninvalid create\",\n\"organism_id\":13596),\n\"reason\":\n\"create\nfail\",\n\"status\":\n\"PENDING\",\n\"requester_id\":10,\n\"reviewer_id\":null,\n\"review_comment\":null,\n\"created_at\":\n\"2026-05-20T13:01:38\",\n\"reviewed_at\":null}\n"}",
    "request_id": 21,
    "review": {
      "status": 500,
      "body": "Internal Server Error"
    },
    "counts": {
      "sequence": 0,
      "review": 0,
      "log": 0
    }
  }
}
```

16	CREATE	DEMO_CREATE_BAD_001	PENDING	10	create fail
----	--------	---------------------	---------	----	-------------

6 存储过程控制下的更新操作（18 分）

6.1 功能描述（1 分）

管理员审批通过 UPDATE 请求后，通过存储过程统一执行跨表更新（序列表 + 特征表），并记录操作日志。

6.2 存储过程功能描述（1 分）

`sp_update_sequence_reviewed` 对目标 accession 执行受控更新：

1. 校验 accession 是否存在；
2. 更新 Sequence 表的基础字段；
3. 根据是否已有 `key='gene'` 的 Feature 记录决定更新或插入；
4. 写入 `sequence_operation_log` 审计日志。

6.3 涉及的关系表（2 分）

Sequence、Feature、`sequence_operation_log`

6.4 表连接涉及字段（1 分）

`Feature.accession = Sequence.accession`

字段	规则
Sequence.version/locus/definition/ organism_id/mol_type/sequence	使用 COALESCE(参数, 原值), 仅更新传入字段
Feature(gene/product/location/note)	若存在 key='gene' 则更新, 否则插入新行
sequence_operation_log	每次成功更新, 记录 update_approved 审计日志

表 5: 更新操作字段规则

6.5 更改字段 (2 分)

6.6 更新代码 (3 分)

```

1 # apps/backend/app/services/sequence_request_service.py
2 CALL sp_update_sequence_reviewed(
3     :accession,
4     :version,
5     :locus,
6     :definition,
7     :organism_id,
8     :mol_type,
9     :sequence,
10    :feature_gene,
11    :feature_product,
12    :feature_location,
13    :feature_note
14 )

```

6.7 创建存储过程源码 (3 分)

```

1 CREATE PROCEDURE sp_update_sequence_reviewed(
2     IN p_accession VARCHAR(50),
3     IN p_version VARCHAR(50),
4     IN p_locus VARCHAR(50),
5     IN p_definition VARCHAR(255),
6     IN p_organism_id INT,
7     IN p_mol_type VARCHAR(50),
8     IN p_sequence LONGTEXT,
9     IN p_gene VARCHAR(100),
10    IN p_product VARCHAR(255),
11    IN p_location VARCHAR(100),
12    IN p_note TEXT
13 )
14 BEGIN
15     DECLARE v_seq_exists INT;

```

```

16
17 SELECT COUNT(*) INTO v_seq_exists
18 FROM `Sequence`
19 WHERE accession = p_accession;
20
21 IF v_seq_exists = 0 THEN
22     SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT =
23     → 'accession does not exist';
24 END IF;
25
26 -- 更新 Sequence 表（仅更新非空参数）
27 UPDATE `Sequence`
28 SET
29     version = COALESCE(p_version, version),
30     locus = COALESCE(p_locus, locus),
31     definition = COALESCE(p_definition, definition),
32     organism_id = COALESCE(p_organism_id, organism_id),
33     mol_type = COALESCE(p_mol_type, mol_type),
34     sequence = COALESCE(p_sequence, sequence)
35 WHERE accession = p_accession;
36
37 -- Feature 更新或插入逻辑（key='gene'）
38 IF p_gene IS NOT NULL THEN
39     INSERT INTO `Feature` (accession, `key`, gene, product, location,
40     → note)
41     VALUES (p_accession, 'gene', p_gene, p_product, p_location, p_note)
42     ON DUPLICATE KEY UPDATE
43     gene = VALUES(gene),
44     product = VALUES(product),
45     location = VALUES(location),
46     note = VALUES(note);
47 END IF;
48
49 -- 写入操作日志
50 INSERT INTO `sequence_operation_log` (accession, operate_type,
51     operator_name, operate_desc)
52 VALUES (p_accession, 'update_approved', 'review_engine',
53     'sequence updated by approved change request');
54 END;

```

6.8 存储过程执行源码（1 分）

```

1 CALL sp_update_sequence_reviewed(
2     'NC_XXXXX',
3     '2.0',

```

```

4     NULL,
5     'updated definition',
6     NULL,
7     NULL,
8     'ATCGATCGATCGATCGATCG',
9     'rbcl',
10    'updated product',
11    'chr1:1-20(plus)',
12    'updated by workflow'
13 );

```

6.9 程序演示（4 分）

6.9.1 合法更新演示（2 分）

```

1  # 1) 提交 UPDATE 请求（仅传部分字段，验证 COALESCE 语义）
2  curl -s -X POST "http://127.0.0.1:8001/api/v1/sequence-requests" \
3  -H "Authorization: Bearer $USER_TOKEN" \
4  -H "Content-Type: application/json" \
5  -d '{
6      "action_type": "UPDATE",
7      "target_accession": "DEMO_CREATE_OK_001",
8      "payload_json": {
9          "definition": "Updated by sp_update_sequence_reviewed demo",
10         "mol_type": "mRNA",
11         "feature_gene": "demo_gene_upd",
12         "feature_product": "demo_product_upd",
13         "feature_location": "10..120",
14         "feature_note": "note from procedure"
15     },
16     "reason": "demo update success"
17 }'
18
19 # 2) 管理员审批通过
20 curl -s -X POST \
21     ↪ "http://127.0.0.1:8001/api/v1/admin/sequence-requests/{request_id}/review" \
22     ↪ \
23     ↪ -H "Authorization: Bearer $ADMIN_TOKEN" \
24     -H "Content-Type: application/json" \
25     -d '{"decision": "APPROVE", "review_comment": "approve update demo"}'

```

```
{
  "submit": {
    "status": 201,
    "body": "{\"id\":22,\"action_type\":\"UPDATE\",\"target_accession\":\"DEMO_CREATE_OK_001\",\"payload_json\":{\"mol_type\":\"mRNA\",\"definition\":\"Updated by sp_update_sequence_reviewed demo\",\"feature_gene\":\"demo_gene_upd\",\"feature_note\":\"demo note from procedure\",\"feature_product\":\"demo_product_upd\",\"feature_location\":\"10..120\"},\"reason\":\"update ok\",\"status\":\"PENDING\",\"requester_id\":10,\"reviewer_id\":null,\"review_comment\":null,\"created_at\":\"2026-05-20T13:01:38\",\"reviewed_at\":null}\""
  },
  "request_id": 22,
  "review": {
    "status": 200,
    "body": "{\"request_id\":22,\"status\":\"APPROVED\",\"execution_summary\":\"UPDATE executed for accession DEMO_CREATE_OK_001.\"}"
  },
  "sequence": [],
  "feature": [],
  "log": []
}
```

17 UPDATE DEMO_CREATE_OK_001

APPROVED

10

update ok

real-shot run

6.9.2 非法更新演示（2 分）

```
1 # target_accession 不存在，存储过程应抛错
2 curl -s -X POST "http://127.0.0.1:8001/api/v1/sequence-requests" \
3 -H "Authorization: Bearer $USER_TOKEN" \
4 -H "Content-Type: application/json" \
5 -d '{
6   "action_type": "UPDATE",
7   "target_accession": "DEMO_NO_SUCH_ACCESSION_001",
8   "payload_json": {
9     "definition": "should fail"
10  },
11   "reason": "demo update fail"
12 }'
13
14 curl -s -X POST '
15   ↪ "http://127.0.0.1:8001/api/v1/admin/sequence-requests/{request_id}/review"
16   ↪
17   ↪ \
18 -H "Authorization: Bearer $ADMIN_TOKEN" \
19 -H "Content-Type: application/json" \
20 -d '{"decision": "APPROVE", "review_comment": "approve fail-case"}'
```

```
{
  "submit": {
    "status": 201,
    "body": "{\"id\":23,\"action_type\":\"UPDATE\",\"target_accession\":\"DEMO_NO_SUCH_ACCESSION_001\",\"payload_json\":{\"definition\":\"should fail\"},\"reason\":\"update fail\",\"status\":\"PENDING\",\"requester_id\":10,\"reviewer_id\":null,\"review_comment\":null,\"created_at\":\"2026-05-20T13:01:38\",\"reviewed_at\":null}"
  },
  "request_id": 23,
  "review": {
    "status": 500,
    "body": "Internal Server Error"
  },
  "counts": {
    "sequence": 0,
    "feature": 0,
    "log": 0
  }
}
```

18	UPDATE	DEMO_NO_SUCH_ACCESSION_001	PENDING	10	update fail
----	--------	----------------------------	---------	----	-------------

7 含有视图的查询操作（15 分）

7.1 操作功能描述（1 分）

系统首页检索和详情页查询均基于视图进行数据聚合展示，支持按关键词、状态、属名、分页、排序查询。

7.2 视图功能描述（1 分）

`v_nucleotide_sequence_info` 将 `Sequence` 与物种、分类、审核状态、操作统计等信息整合为统一查询界面，减少应用层重复 JOIN 操作的复杂度。

7.3 涉及的关系表（2 分）

`Sequence`、`Organism`、`Taxon`、`sequence_review`、`sequence_operation_log`

7.4 表连接字段（1 分）

- `Organism.organism_id = Sequence.organism_id`
- `Taxon.taxon_id = Organism.taxon_id`
- `sequence_review.accession = Sequence.accession`
- `op(accession) = Sequence.accession` (op 为日志聚合子查询)

7.5 创建视图代码（3 分）

```
1 CREATE VIEW v_nucleotide_sequence_info AS
2 SELECT
```

```

3      s.accession, s.version, s.locus,
4      s.definition, s.organism_id,
5      o.scientific_name, t.genus,
6      COALESCE(sr.seq_status, 1) AS seq_status,
7      CASE COALESCE(sr.seq_status, 1)
8          WHEN 0 THEN 'pending'
9          WHEN 1 THEN 'approved'
10         WHEN 2 THEN 'rejected'
11         ELSE 'unknown'
12     END AS seq_status_desc,
13     COALESCE(sr.submitter, 'seed_script') AS submitter,
14     COALESCE(sr.submit_time, CURRENT_TIMESTAMP) AS submit_time,
15     s.length AS seq_length,
16     CONCAT(LEFT(COALESCE(s.sequence, ''), 20), '..') AS seq_content_simple,
17     COALESCE(op.operate_count, 0) AS operate_count,
18     op.last_operate_time
19 FROM `Sequence` AS s
20 LEFT JOIN `Organism` AS o ON o.organism_id = s.organism_id
21 LEFT JOIN `Taxon` AS t ON t.taxon_id = o.taxon_id
22 LEFT JOIN `sequence_review` AS sr ON sr.accession = s.accession
23 LEFT JOIN (
24     SELECT accession, COUNT(*) AS operate_count, MAX(operate_time) AS
25         ↳ last_operate_time
26     FROM `sequence_operation_log`
27     GROUP BY accession
28 ) AS op ON op.accession = s.accession;

```

7.6 查询代码（3 分）

```

1  SELECT accession, version, locus, scientific_name, genus,
2      seq_status_desc, seq_length, submit_time, operate_count
3  FROM v_nucleotide_sequence_info
4  WHERE (
5      accession LIKE :keyword OR
6      locus LIKE :keyword OR
7      definition LIKE :keyword OR
8      scientific_name LIKE :keyword
9  )
10 AND seq_status_desc = :status
11 AND genus = :genus
12 ORDER BY submit_time DESC
13 LIMIT :limit OFFSET :offset;

```

7.7 程序演示（4 分）

```
1 # 示例：关键词 + 状态 + 属名 + 分页 + 排序
2 curl -s 'http://127.0.0.1:8001/api/v1/sequences/search?keyword=DEMO&status=approved&genus=Nicotiana'

1 USE purple_bank;
2 SELECT accession, version, locus, scientific_name, genus,
3        seq_status_desc, seq_length, submit_time, operate_count
4 FROM v_nucleotide_sequence_info
5 WHERE (
6     accession LIKE '%DEMO%' OR
7     locus LIKE '%DEMO%' OR
8     definition LIKE '%DEMO%' OR
9     scientific_name LIKE '%DEMO%'
10 )
11 AND seq_status_desc = 'approved'
12 AND genus = 'Nicotiana'
13 ORDER BY submit_time DESC
14 LIMIT 5 OFFSET 0;
```

DEMO_CREATE_OK_001

Updated by sp_update_sequence_reviewed demo

Approved Nicotiana tabacum Length: 20

Sequence

ATCGATCGATCGATCGATCG

Features (1)

ID	Key	Location	Gene	Product
27273	gene	10..120	demo_gene_upd	demo_product_upd

7.8 备注

API 接口地址：GET /api/v1/sequences/search

DEMO_DELETE_OK_001

Demo rollback sample

Approved

Nicotiana tabacum

Length: 24

Sequence

ATCGATCGATCGATCGATCGATCG

Features (1)

ID	Key	Location	Gene	Product
27272	gene	1..24	demo_gene_del	demo_product_del